

===== SEMAINE 1 =====

Installer Maven

```
sudo apt install maven -y
mvn -version
```

Compiler des dossiers

```
mvn clean install
```

Execution des exemples

Les cmd mvn se lance depuis le dossier **/CloudSIM-TSP/modules/cloudsim-examples** Contenant fichier **pom.xml**

Dans le dossier des exemples

```
cd ~/Documents/TSP_S2/Cassiopee/CloudSIM-TSP/modules/cloudsim-examples
```

Exemple 1

```
mvn exec:java -
Dexec.mainClass="org.cloudbus.cloudsim.examples.CloudSimExample1"
```

les autres exemples

```
for i in 2 3 4 5 6 7 8 9; do
  echo
  echo "-      CloudSimExample$i      -"
  echo
  mvn exec:java -
  Dexec.mainClass="org.cloudbus.cloudsim.examples.CloudSimExample$i"\
  2>/dev/null | grep -A 200 "===== OUTPUT ====="
done
```

COMPRENDRE L'OUTILS ET COMPOSANTS AVANT DE DEMARRER

5 Concepts fondamentaux

1. Cloudlet (La tâche)

C'est **une tâche à exécuter** (comme un programme, un calcul, un traitement de données)

Dans la vraie vie : C'est un job qu'un utilisateur soumet au cloud (ex: "traite ce fichier Excel", "rends cette vidéo")

Paramètres

Longueur (MI) = Million d'Instructions : plus c'est grand, plus la tâche est longue

2. VM

Un "ordinateur virtuel" qui exécute les tâches

Paramètres :__

MIPS = Million d'Instructions Par Seconde : C'est la puissance de la VM

Plus le MIPS est élevé, plus la VM est rapide

3. Host (Serveur physique)

C'est quoi ? Le vrai serveur dans le datacenter

Paramètres

Nb de PE (Processing Element) = Nombre de Cœurs du Processeur (CPU)

Un host peut héberger plusieurs VMs

4. Datacenter

C'est Le bâtiment avec tous les serveurs

Paramètres

Combien de hosts, comment allouer les ressources ?

5. Broker (Le courtier)

L'intermédiaire entre l'utilisateur et le cloud

Dans la vraie vie : La console AWS ou Azure qu'on utilise pour lancer des VMs

Rôle : Il dit "je veux ces VMs" puis "je veux exécuter ces tâches sur ces VMs"

Tableau Récapitulatif

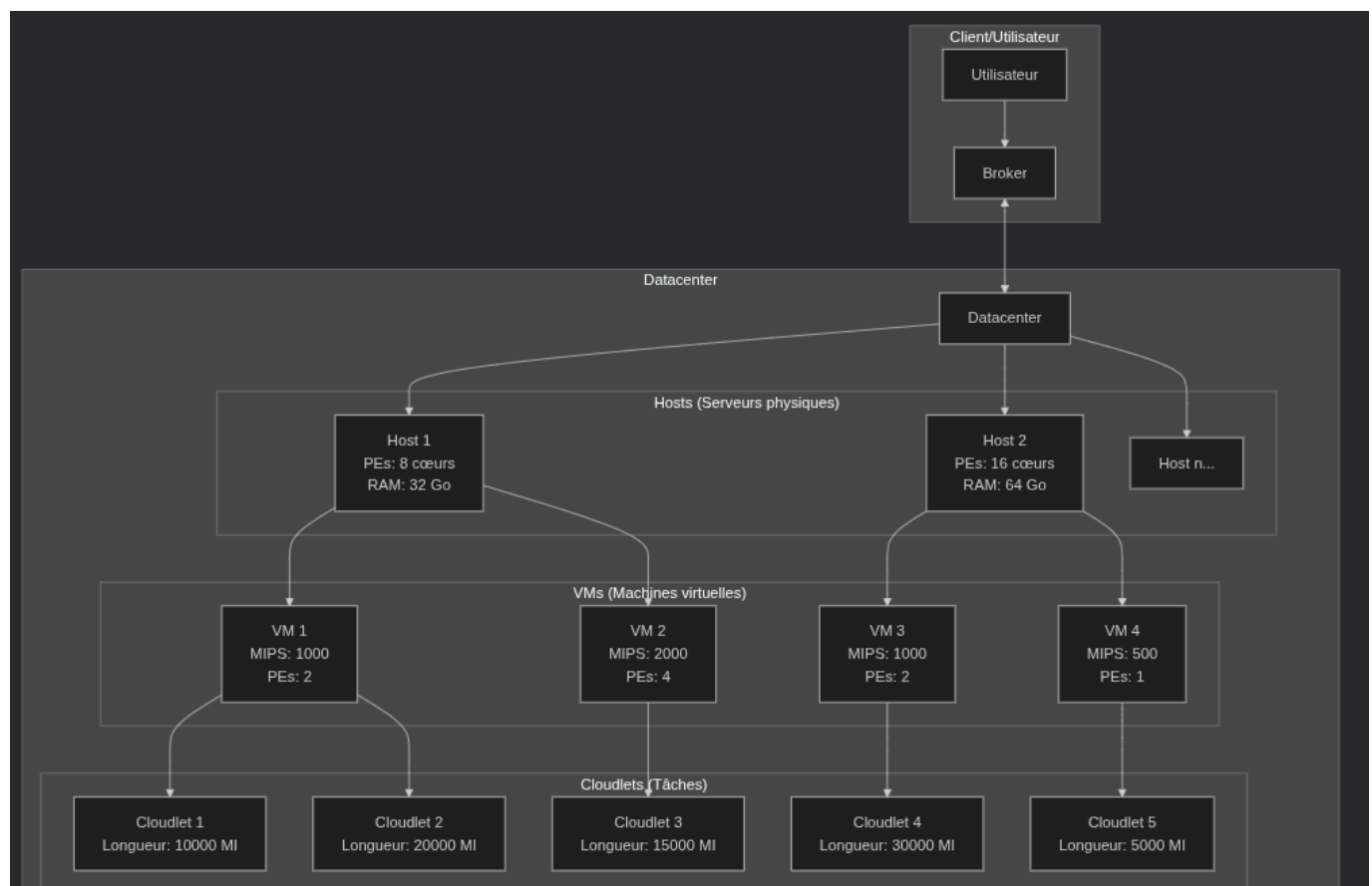
1. Ce que c'est (en une phrase)

CloudSim est un **framework Java** qui simule l'exécution de tâches dans des datacenters virtuels, sans avoir besoin d'une vraie infrastructure cloud.

2. Les 5 concepts fondamentaux

Concept	Ce que ça représente dans la vraie vie	Paramètres clés
Cloudlet	Une tâche applicative (ex: traitement de données)	Longueur (MI), nb cœurs, taille I/O
VM	Une machine virtuelle sur un serveur	MIPS, RAM, BW, nb cœurs
Host	Un serveur physique dans le datacenter	Nb de PE, RAM, stockage, PowerModel
Datacenter	L'ensemble du datacenter	Nb d'hôtes, politique d'allocation
Broker	Le gestionnaire cloud côté client	Soumet les VMs et les Cloudlets

Architecture Nominal



Métriques de performances

Performance → Finish Time de chaque Cloudlet (temps d'exécution) **Énergie Consommée** → `getEnergyConsumption()` sur les hôtes (en Joules ou kWh)

Finish Time

Il est extrait directement de l'objet Cloudlet à la fin de la simulation.

On peut accéder à ce résultat via :

```
cloudlet.getFinishTime();
```

Calcul de la consommation d'énergie

Chaque Host possède un modèle énergétique (par exemple, PowerModel). Ce modèle calcule la consommation en fonction de la charge CPU réelle pendant la simulation.

On obtient la consommation énergétique avec :

```
host.getEnergyConsumption(); // Joules ou kWh selon modèle
```

Cette valeur est une somme sur toute la durée de la simulation.

===== SEMAINE 2 =====

OBJECTIF : Comprendre Comment une tâche est exécutée dans CloudSim, de A à Z

1. datacenterBroker

L'intermédiaire entre l'utilisateur et le datacenter. **Tu lui donnes des VMs et des Cloudlets, il gère tout le reste automatiquement.**

Creation

```
DatacenterBroker broker = new DatacenterBroker("Broker_0");
```

Methodes utiles

```
//1. Soumettre les VMs au datacenter  
broker.submitGuestList(vmList);  
  
// 2. Soumettre les Cloudlets  
broker.submitCloudletList(cloudletList);  
  
// 3. Forcer une tâche sur une VM spécifique (optionnel)  
broker.bindCloudletToVm(cloudletId, vmId);
```

scenario

4. Ce qui se passe automatiquement en interne

1. broker démarre
2. découvre les datacenters disponibles
3. crée les VMs sur le premier datacenter disponible
4. une fois les VMs créées → soumet les Cloudlets
(par défaut : round-robin sur les VMs)
5. quand un Cloudlet finit → le récupère dans cloudletReceivedList
6. quand tout est fini → détruit les VMs → fin de simulation

Important : Par défaut le broker distribue les Cloudlets en round-robin sur les VMs. Pour un contrôle précis on utilisera `bindCloudletToVm()`.

Une Methode pour recuperer les résultats

```
// Récupérer tous les Cloudlets terminés avec leurs métriques  
broker.getCloudletReceivedList()  
  
// Exemple de boucle de collecte des résultats :  
for (Cloudlet c : broker.getCloudletReceivedList()) {  
    double temps = c.getExecFinishTime() - c.getExecStartTime();  
    String statut = c.getStatus().toString();  
}
```

2. Datacenter.java

Le bâtiment qui contient les serveurs. Il reçoit les événements et les distribue aux bons hôtes et VMs.

Parametres

```
new Datacenter(  
    name,                // nom ex: "Datacenter_1"  
    characteristics,     // description des hôtes (CPU, RAM, coût...)  
    vmAllocationPolicy,  // comment placer les VMs sur les hôtes ← important  
    storageList,         // stockage disponible  
    schedulingInterval   // fréquence de mise à jour (ex: 0.1 seconde)  
)
```

Le datacenter ne contient pas directement les hôtes. Il les contient via **DatacenterCharacteristics**. C'est un niveau d'indirection à retenir.

Focus

vmAllocationPolicy : Décide de comment les VMs sont réparties sur les hôtes, ce qui impacte directement la **charge CPU** et donc **l'énergie**.

Remarque

Datacenter.java ne mesure pas l'énergie. Pour le projet on utilisera sa classe fille :

PowerDatacenter.java qui mesure l'énergie

3. Powerdatacenter.java

C'est **Datacenter.java + la mesure d'énergie**. C'est cette classe qu'on utiliseras dans tous les scénarios.

Parametres

```
new PowerDatacenter(  
    name,  
    characteristics,  
    vmAllocationPolicy, // impact sur l'énergie  
    storageList,  
    schedulingInterval  // plus il est petit, plus la mesure est précise  
)
```

Metrique Energie

```
powerDatacenter.getPower() // énergie totale consommée en Watt.seconde
```

Autres

Élément	Pourquoi pertinent
<code>getPower()</code>	Ta métrique énergie en W-sec
<code>schedulingInterval</code>	Précision des mesures
<code>disableMigrations</code>	Activer/désactiver pour comparer l'impact énergie
<code>getMigrationCount()</code>	Mesurer le coût en migrations

4. host.java

Un serveur physique dans le datacenter. C'est lui qui héberge les VMs, et c'est **sa charge CPU qui détermine l'énergie consommée**.

Parametres

```
new Host(
    id,
    ramProvisioner, // gestion de la RAM ex: new
    RamProvisionerSimple(ram)
    bwProvisioner, // gestion bande passante ex: new
    BwProvisionerSimple(bw)
    storage, // stockage en MB
    peList, // liste des cœurs CPU ← impact direct sur
    performance
    vmScheduler // comment les VMs se partagent les cœurs
)
```

Param. impactant Energie et Performance

PE = Un cœur physique avec une capacité en MIPS

```
List<Pe> peList = new ArrayList<>();

peList.add(new Pe(0, new PeProvisionerSimple(1000))); // 1 cœur à 1000 MIPS

peList.add(new Pe(1, new PeProvisionerSimple(1000))); // 2e cœur
```

NB : Chaque ligne ajoute un (01) coeurs

VmScheduler = Partage des cœurs entre VMs

```
new VmSchedulerTimeShared(peList) : Les VMs partagent les cœurs et
                                     L'exécution se fait en simultané
```

Chaque VM reçoit une fraction des MIPS
(ex: 2 VMs sur 1000 MIPS → 500 MIPS
chacune)

`new VmSchedulerSpaceShared(peList)` : Chaque VM obtient un cœur dédié
exclusif

- Si un cœur est disponible → VMs
créées immédiatement
- Si aucun cœur disponible → VM en
attente (même datacenter)
→ OU
broker cherche ailleurs

Méthodes Utiles

```
host.getTotalMips()           // capacité totale du serveur
host.getAvailableMips()       // MIPS encore disponibles
host.getNumberOfFreePes()     // cœurs libres
host.getGuestList()           // liste des VMs hébergées
```

Remarques

`Host.java` seul ne mesure pas l'énergie. Il faut sa classe fille : `PowerHost` contient le `PowerModel` (consommation physique du serveur).

5. PowerHost.java

C'est **Host.java + un modèle de consommation électrique**. C'est la classe la plus importante pour mesurer l'énergie par serveur.

Creation : un seul paramètre en plus par rapport à Host

```
new PowerHost(
    id,
    ramProvisioner,
    bwProvisioner,
    storage,
    peList,
    vmScheduler,
    powerModel           // ← LE paramètre clé : modèle de consommation
    électrique
)
```

Le PowerModel : pièce centrale pour l'énergie

C'est lui qui définit **combien de Watts consomme le serveur selon sa charge CPU**.

CloudSim fournit plusieurs modèles prêts à l'emploi dans `power/models/` :

Méthodes utiles

```
// Consommation instantanée en Watts selon la charge actuelle
host.getPower()

// Consommation pour une charge donnée (entre 0 et 1)
host.getPower(0.75) // consommation à 75% de charge CPU

// Historique de charge CPU ← utile pour tracer des courbes
host.getUtilizationHistory()

// Charge CPU actuelle (héritée de HostDynamicWorkload)
host.getUtilizationOfCpu()
```

Quelques modèles de consommation

Tous les modèles répondent à la même question : **"Si mon serveur est chargé à X%, combien de Watts consomme-t-il ?"**

Formule générale

$$\text{Consommation} = \text{idle} + (\text{maxPower} - \text{idle}) \times f(\text{utilisation})$$

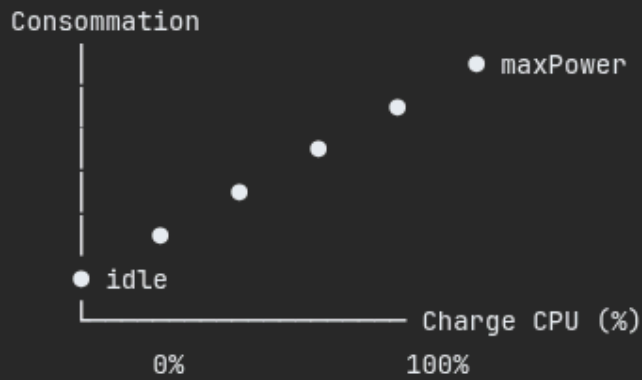
Ce qui change entre les modèles, c'est la fonction $f()$: la forme de la courbe.

Paramètres Communs

```
new PowerModelLinear(maxPower, staticPowerPercent)
// maxPower          = consommation à 100% de charge (ex: 200W)
// staticPowerPercent = % consommé à vide (ex: 0.5 = 50% → 100W idle)
```

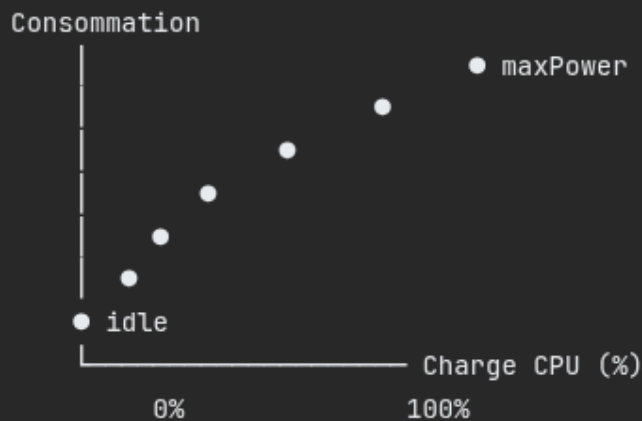
Les 4 philosophies

PowerModelLinear



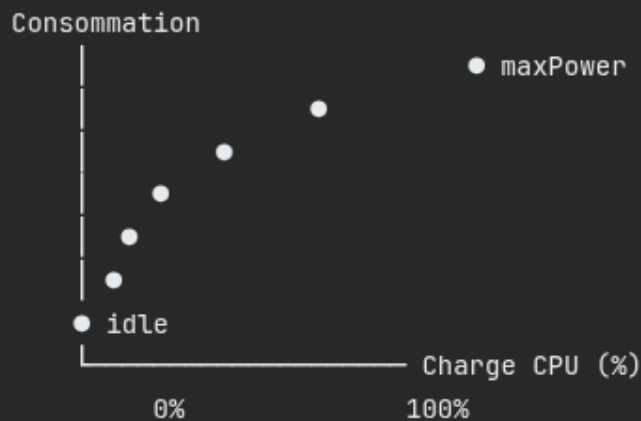
Philosophie : chaque % de charge supplémentaire coûte **toujours le même prix** en énergie. Simple, prévisible. Pas très réaliste mais parfait pour comprendre.

PowerModelSquare



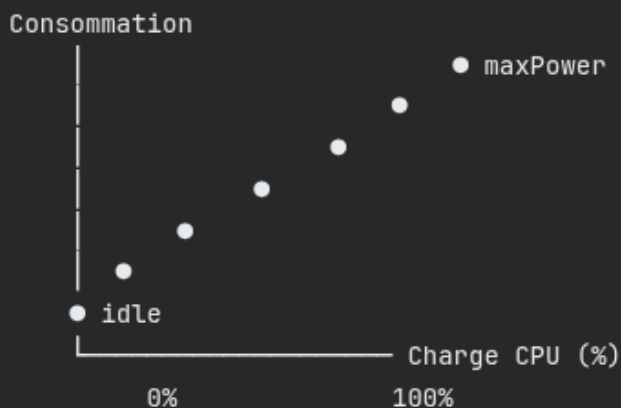
Philosophie : la consommation s'**emballe** à forte charge. Modélise un serveur qui chauffe et consomme beaucoup plus quand il est sollicité intensément.

PowerModelSqrt



Philosophie : la consommation monte vite au début puis se stabilise. Modélise un serveur efficace — même à forte charge, il ne consomme pas beaucoup plus qu'à charge moyenne.

PowerModelCubic



Philosophie : la consommation reste quasi-plate jusqu'à forte charge, puis explose. Modélise un serveur très efficace en dessous de 70-80% mais très gourmand au-delà.

6. Vm.java

Une machine virtuelle qui tourne sur un hôte physique. Elle **reçoit les Cloudlets et les exécute selon une politique d'ordonnancement**.

Création

```
new Vm(
    id,                // identifiant unique
    userId,            // id du broker propriétaire
    mips,              // puissance de calcul par cœur ← impact direct
    temps d'exécution
    numberOfPes,       // nb de cœurs ← impact charge CPU → énergie
```

```

    ram,                // RAM en MB
    bw,                 // bande passante
    size,               // taille image disque en MB
    vmm,                // "Xen" par convention
    cloudletScheduler   // ← politique d'ordonnancement des tâches
)

```

Focus

```

mips,                  // puissance de calcul par cœur ← impact direct temps
d'exécution__

numberOfPes,           // nb de cœurs ← impact charge CPU → énergie__

cloudletScheduler      // ← politique d'ordonnancement des tâches__

```

Temps d'exécution = cloudletLength / mips

CloudletScheduler : paramètre crucial

C'est lui qui définit **comment la VM gère plusieurs Cloudlets en même temps** :

```

new CloudletSchedulerTimeShared()
// Tous les Cloudlets s'exécutent en parallèle
// Chacun reçoit une fraction du MIPS → tous finissent plus tard
// Charge CPU élevée en permanence

new CloudletSchedulerSpaceShared()
// Les Cloudlets s'exécutent un par un
// Chacun reçoit 100% du MIPS → file d'attente possible
// Charge CPU plus variable

```

Méthodes utiles

```

1. Charge CPU de la VM à un instant donné (utilisée par PowerHost)

vm.getTotalUtilizationOfCpu(time)    // entre 0 et 1
vm.getTotalUtilizationOfCpuMips(time) // en MIPS absolus

2. Ressources allouées réellement par l'hôte

vm.getCurrentAllocatedMips()          // MIPS effectivement alloués
vm.getCurrentAllocatedRam()           // RAM effectivement allouée

3. État de la VM

```

```
vm.isInMigration()           // ← utile Phase 3 pour suivre les  
migrations  
vm.getMips()                 // puissance configurée  
vm.getNumberOfPes()          // nb de cœurs
```

c'est la VM qui remonte la charge CPU à l'hôte, qui calcule ensuite l'énergie via le PowerModel.

7. Cloudlet.java

Un Cloudlet = une tâche applicative à exécuter dans le cloud. Dans la vraie vie c'est par exemple : un job de calcul, un traitement de données, une requête web.

cloudletLength représente la quantité de travail d'une tâche en MI (Million Instructions).

1 MI = 1 000 000 instructions à exécuter (C'est la taille de la tâche que la machine virtuelle doit traiter).

MIPS (Million Instructions Per Second)

MIPS = puissance de calcul de la machine (1 MIPS = 1 million d'instructions par seconde)

Temps d'execution = (MI / MIPS)

Charge Totale = cloudletLength × numberOfPes (MI)

UtilizationModel

C'est ce qui définit comment la tâche consomme les ressources.

3 Types

```
// 100% du CPU tout le temps (le plus simple)  
new UtilizationModelFull()  
  
// Un pourcentage fixe (ex: 50% du CPU)  
new UtilizationModelStochastic()  
  
// Variable dans le temps (basé sur des traces réelles PlanetLab)  
new UtilizationModelPlanetLabInMemory(fichier)
```

Paramètres (pour créer un Cloudlet)

```
new Cloudlet(  
    cloudletId,          // identifiant unique  
    cloudletLength,      // taille de la tâche en MI (Million Instructions)  
    pesNumber,           // nb de cœurs CPU nécessaires  
    cloudletFileSize,    // taille des données d'entrée (bytes)  
    cloudletOutputSize, // taille des données de sortie (bytes)  
    utilizationModelCpu, // comment la tâche utilise le CPU  
    utilizationModelRam, // comment la tâche utilise la RAM  
    utilizationModelBw   // comment la tâche utilise la bande passante  
)
```

Métriques

```
cloudlet.getExecStartTime() // quand la tâche a démarré  
cloudlet.getExecFinishTime() // quand la tâche s'est terminée  
cloudlet.getActualCPUTime() // temps CPU réellement consommé  
cloudlet.getStatus() // SUCCESS, FAILED, CANCELED...  
cloudlet.getWaitingTime() // temps passé en file d'attente
```

Focus

```
// TEMPS D'EXÉCUTION  
cloudlet.getExecFinishTime() - cloudlet.getExecStartTime()  
  
// STATUT (la tâche a-t-elle bien tourné ?)  
cloudlet.getStatus() == SUCCESS
```

```
cloudlet.getWallClockTime() ----> temps total (attente + exécution)
```

POLITIQUES D'ORDONNANCEMENT

Niveau 1 - **VmScheduler** : partage des cœurs physiques entre VMs

Question : comment un hôte physique partage ses cœurs entre plusieurs VMs ?

VmSchedulerTimeShared

Hôte : 2 cœurs à 1000 MIPS chacun = 2000 MIPS total

3 VMs demandent chacune 1000 MIPS

- Chaque VM reçoit 666 MIPS ($2000 \div 3$)
- Les VMs tournent TOUTES en même temps mais plus lentement
- Charge CPU de l'hôte : élevée et constante

VmSchedulerSpaceShared

Hôte : 2 cœurs à 1000 MIPS chacun

3 VMs demandent chacune 1000 MIPS

- VM1 reçoit cœur 1 complet (1000 MIPS)
- VM2 reçoit cœur 2 complet (1000 MIPS)
- VM3 attend qu'un cœur se libère ← file d'attente
- Charge CPU : 100% sur les cœurs utilisés ↓ 0% sinon

Niveau 2 - CloudletScheduler : partage des ressources VM entre Cloudlets

Question : comment une VM partage ses ressources entre plusieurs Cloudlets ?

CloudletSchedulerTimeShared

VM : 1000 MIPS

3 Cloudlets de 10 000 MI chacun

- Chaque Cloudlet reçoit 333 MIPS
- Tous s'exécutent en parallèle
- Temps de fin : $10\ 000 \div 333 = 30$ secondes chacun
- Charge CPU de la VM : 100% en permanence

CloudletSchedulerSpaceShared

VM : 1000 MIPS

3 Cloudlets de 10 000 MI chacun

- Cloudlet 1 s'exécute seul à 1000 MIPS → fini en 10 sec
- Cloudlet 2 démarre → fini à 20 sec
- Cloudlet 3 démarre → fini à 30 sec
- Charge CPU : 100% puis 0% à la fin

Makespan = temps entre le début du 1er Cloudlet et la fin du dernier : c'est la métrique de performance globale.

Remarques

Processor Sharing = C'est une variante de **TimeShared** mais plus fine.

TimeShared classique : Divise le MIPS également entre tous les Cloudlets

Processor Sharing :

- Divise proportionnellement selon la demande de chaque Cloudlet
- Plus équitable, plus proche du comportement réel des OS

Dans CloudSim c'est CloudletSchedulerDynamicWorkload

===== SEMAINE 3 =====

CloudSimExample1.java

Ceci est un exemple simple de simulation avec CloudSim pour modéliser l'exécution de tâches dans un cloud.

Ce fichier :

- Initialise l'environnement de simulation et crée un Datacenter et un DatacenterBroker.
- Configure une machine virtuelle (VM) et deux cloudlets (tâches à exécuter).
- Lance la simulation puis affiche les résultats : VM utilisée, temps d'exécution, début et fin des tâches.

1. Initialisation de Cloudsim

```
int num_user = 1; // 1 seul
utilisateur/broker
boolean trace_flag = false; // pas de log détaillé
CloudSim.init(num_user, calendar, trace_flag);
```

2. Creation du datacenter

```
Datacenter datacenter0 = createDatacenter("Datacenter_0");
```

Dans **createDatacenter()** :

```
// 1 seul cœur physique à 1000 MIPS
peList.add(new Pe(new PeProvisionerSimple(1000)));

// 1 seul hôte
```

```

new Host(
    new RamProvisionerSimple(2048),        // 2 GB RAM
    new BwProvisionerSimple(10000),        // 10 Gbps
    10000000,                               // 1 TB stockage
    peList,
    new VmSchedulerTimeShared(peList)      // partage TimeShared
)

// Datacenter basique : Pas de mesure énergie ici (Car Non Using de
// PowerDatacenter)
new Datacenter(name, characteristics,
    new VmAllocationPolicySimple(hostList), ...)

```

3. Creation du Broker :

```
broker = new DatacenterBroker("Broker");
```

4. Création de la VM

```

int mips = 1000;                // puissance de calcul
int pesNumber = 1;              // 1 seul cœur
int ram = 512;                  // 512 MB RAM
long bw = 1000;                 // bande passante
new CloudletSchedulerTimeShared() // ordonnancement TimeShared

```

5. Création du Cloudlet

```

long length = 400000;           // 400 000 MI
int pesNumber = 1;              // 1 cœur
new UtilizationModelFull() // utilise 100% des ressources

```

Tests

Remarques : Recompiler après chaque modif. comme ci-dessous :

Commande

```

cd ~/Documents/TSP_S2/Cassiopee/CloudSIM-TSP
mvn clean install -DskipTests -q && cd modules/cloudsim-examples && mvn
exec:java -
Dexec.mainClass="org.cloudbus.cloudsim.examples.CloudSimExample1"
2>/dev/null | grep -A 200 "===== OUTPUT ====="

```

Test_0 : Execution de l'exemple 1

```
francy@Ubuntu:~/Documents/TSP_S2/Cassiopee/CloudSIM-TSP/modules/cloudsim-examples$ mvn exec:java -Dexec.mainClass="org.cloudbus.
cloudsim.examples.CloudSimExample1"
2>/dev/null | grep -A 200 "===== OUTPUT ====="
===== OUTPUT =====
Cloudlet ID   STATUS   Data center ID   VM ID   Time   Start Time   Finish Time
0             SUCCESS   2               0       400    0,01         400,01
CloudSimExample1 finished!
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.169 s
[INFO] Finished at: 2026-03-16T01:22:59+01:00
[INFO] -----
```

Variations des Paramètres

- Test_1 : Variation de la Puissance du VM.mips= 2000, 500, 100
- Test_2 : Variation de la taille du Cloudlet

Leçons

1. $MIPS_{VM} \leq MIPS_{H\hat{O}TE}$ (Sinon, Aucun Cloudlet sera crée)
2. REGLE : Temps = cloudletLength ÷ mips
3. Le Temps d'exécution varie en fonction de ces paramètres ci-dessous :
 - Cloudlet :
 - Taille
 - Nombre
 - VM et Serveur : MIPS

Test_3 : Cloudlets Identiques + TimeShared(new CloudletSchedulerTimeShared())

Paramètres = 2 Cloudlets de 400 000 MI sur 1 VM à 1000 MIPS.

"TimeShared" --> Partage égal des mips entre Cloudlets : $1000 \div 2 = 500$ MIPS
Et les deux s'exécutent en parallèle

Sortie

```
francy@Ubuntu:~/Documents/TSP_S2/Cassiopee/CloudSIM-TSP/modules/cloudsim-examples$ cd ~/Documents/TSP_S2/Cassiopee/CloudSIM-TSP
&& mvn clean install -DskipTests -q && cd modules/cloudsim-examples && mvn exec:java -Dexec.mainClass="org.cloudbus.cloudsim.exa
mples.CloudSimExample1" 2>/dev/null | grep -A 200 "===== OUTPUT ====="
===== OUTPUT =====
Cloudlet ID   STATUS   Data center ID   VM ID   Time   Start Time   Finish Time
1            SUCCESS    2                0       800     0,01        800,01
0            SUCCESS    2                0       800     0,01        800,01
CloudSimExample1 finished!
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 0.855 s
[INFO] Finished at: 2026-03-16T03:07:38+01:00
[INFO] -----
```

Test_4 : 2 Cloudlets Identiques + SpaceShared (CloudletSchedulerSpaceShared())

Le 1er Commence fini, et l'autre debuteAinsi de suite.

Sortie

```
francy@Ubuntu:~/Documents/TSP_S2/Cassiopee/CloudSIM-TSP/modules/cloudsim-examples$ cd ~/Documents/TSP_S2/Cassiopee/CloudSIM-TSP
&& mvn clean install -DskipTests -q && cd modules/cloudsim-examples && mvn exec:java -Dexec.mainClass="org.cloudbus.cloudsim.exa
mples.CloudSimExample1" 2>/dev/null | grep -A 200 "===== OUTPUT ====="
===== OUTPUT =====
Cloudlet ID   STATUS   Data center ID   VM ID   Time   Start Time   Finish Time
1            SUCCESS    2                0       400     0,01        400,01
0            SUCCESS    2                0       400     400,01      800,01
CloudSimExample1 finished!
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 0.942 s
[INFO] Finished at: 2026-03-16T03:28:52+01:00
[INFO] -----
```

Remarques

- SpaceShared est plus efficace en **temps moyen** (600 vs 800 sec) mais crée de l'attente pour certaines tâches.
- TimeShared est plus équitable : tout le monde finit en même temps mais plus tard.

CloudSimExample2.java

Exemple de simulation avec **2 VMs** et **2 Cloudlets** sur **un seul hôte** (serveur), **chaque tâche** (cloudlets) étant explicitement **liée à une VM dédiée**.

Ce fichier :

- Initialise l'environnement de simulation et crée un seul Datacenter avec un hôte à 1000 MIPS et un DatacenterBroker.
- Configure 2 VMs identiques à 250 MIPS chacune avec un ordonnancement TimeShared.

```
Vm vm1 = new Vm(brokerId, mips, pesNumber, ram, bw, size, vmm, new
CloudletSchedulerTimeShared());
```

```
Vm vm2 = new Vm(brokerId, mips, pesNumber, ram, bw, size, vmm, new
CloudletSchedulerTimeShared());
```

- Crée 2 Cloudlets identiques de 250 000 MI et les lie explicitement chacun à une VM via bindCloudletToVm().

```
broker.bindCloudletToVm(cloudlet1.getId(), vm1.getId());
```

- Lance la simulation puis affiche les résultats : **Time**, **Start Time**, **Finish Time**, ...

Tests

Test_0 : Execution

```
cd ~/Documents/TSP_S2/Cassiopee/CloudSIM-TSP
mvn clean install -DskipTests -q && cd modules/cloudsim-examples && mvn
exec:java -
Dexec.mainClass="org.cloudbus.cloudsim.examples.CloudSimExample2"
2>/dev/null | grep -A 200 "===== OUTPUT ====="
```

```
francy@Ubuntu:~/Documents/TSP_S2/Cassiopee/CloudSIM-TSP$ cd ~/Documents/TSP_S2/Cassiopee/CloudSIM-TSP
mvn clean install -DskipTests -q && cd modules/cloudsim-examples && mvn exec:java -Dexec.mainClass="org.cloudbus.cloudsim.examp
es.CloudSimExample2" 2>/dev/null | grep -A 200 "===== OUTPUT ====="
===== OUTPUT =====
Cloudlet ID   STATUS   Data center ID   VM ID   Time   Start Time   Finish Time
0             SUCCESS   2               0       1000   0,01         1000,01
1             SUCCESS   2               1       1000   0,01         1000,01
CloudSimExample2 finished!
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 0.973 s
[INFO] Finished at: 2026-03-16T15:22:02+01:00
[INFO] -----
```

Remarques

Aucune variation importante des paramètres n'a été réalisée dans cet exemple car les conclusions resteraient identiques à celles de l'Exemple1 concernant la relation entre la taille du Cloudlet et la puissance MIPS.

Cependant l'objectif principal de cet exemple est plutôt de montrer **l'exécution parallèle de tâches sur plusieurs machines virtuelles** ainsi que **l'association explicite entre Cloudlets et VMs via le broker**, d'ou les deux (2) tests ci-dessous :

Test_1 : Suppression du Binding + Ajout de 2 autres cloudlets (tâches)

On commente le binding :

```
//broker.bindCloudletToVm(cloudlet1.getCloudletId(),vm1.getId());
//broker.bindCloudletToVm(cloudlet2.getCloudletId(),vm2.getId());
```

Création des Cloudlets et Integration dans la liste :

```
Cloudlet cloudlet3 = new Cloudlet(length, pesNumber, fileSize, outputSize,
                                utilizationModel, utilizationModel,
                                utilizationModel);
cloudlet3.setUserId(brokerId);

Cloudlet cloudlet4 = new Cloudlet(length, pesNumber, fileSize, outputSize,
                                utilizationModel, utilizationModel,
                                utilizationModel);
cloudlet4.setUserId(brokerId);

cloudletList.add(cloudlet3);
cloudletList.add(cloudlet4);
```

Sortie

```
francy@Ubuntu:~/Documents/TSP_S2/Cassiopee/CloudSIM-TSP/modules/cloudsim-examples$ cd ~/Documents/TSP_S2/Cassiopee/CloudSIM-TSP
mvn clean install -DskipTests -q && cd modules/cloudsim-examples && mvn exec:java -Dexec.mainClass="org.cloudbus.cloudsim.exempl
es.CloudSimExample2" 2>/dev/null | grep -A 200 "===== OUTPUT ====="
===== OUTPUT =====
Cloudlet ID      STATUS      Data center ID  VM ID   Time   Start Time   Finish Time
2                SUCCESS      2              0       2000    0,01         2000,01
0                SUCCESS      2              0       2000    0,01         2000,01
3                SUCCESS      2              1       2000    0,01         2000,01
1                SUCCESS      2              1       2000    0,01         2000,01
CloudSimExample2 finished!
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 0.898 s
[INFO] Finished at: 2026-03-16T16:38:06+01:00
[INFO] -----
```

Interprétation

Dans cette expérience, le nombre de cloudlets a été augmenté à quatre alors que seulement deux machines virtuelles sont disponibles.

Le **broker** distribue les tâches entre les VMs selon une politique simple de **type round-robin** (par défaut, car "binding" commenté). Chaque VM reçoit donc deux cloudlets.

Et comme l'ordonnanceur utilisé est **CloudletSchedulerTimeShared**, les tâches exécutées sur une même VM partagent la puissance CPU. **La capacité de calcul** de la VM (250 MIPS) est donc **divisée entre les deux cloudlets**, ce qui entraîne une diminution de la puissance disponible pour chaque tâche et double le temps d'exécution, passant de 1000 s à environ 2000 s.

CloudSimExample3.java

Exemple de simulation avec **deux (02) VMs de puissances différentes sur un datacenter à 2 hôtes**, pour observer l'impact du MIPS sur le temps d'exécution.

Ce fichier :

- Crée un Datacenter avec 2 hôtes identiques à 1000 MIPS chacun.

```
hostList.add(new Host(...peList...)); // Hôte 1 : 1000 MIPS
hostList.add(new Host(...peList2...)); // Hôte 2 : 1000 MIPS
```

- Configure 2 VMs de puissances différentes :

- VM1 : 250 MIPS
- VM2 : 500 MIPS (le double)

```
Vm vm1 = new Vm(brokerId, mips, ...); // mips = 250 MIPS
Vm vm2 = new Vm(brokerId, mips * 2, ...); // mips * 2 = 500 MIPS
```

- Crée 2 Cloudlets identiques de 40 000 MI liés chacun à une VM.
- Pas de mesure d'énergie — uniquement les temps d'exécution.

Test_0 : Execution

```
cd ~/Documents/TSP_S2/Cassiopee/CloudSIM-TSP
mvn clean install -DskipTests -q && cd modules/cloudsim-examples && mvn
exec:java -
Dexec.mainClass="org.cloudbus.cloudsim.examples.CloudSimExample3"
2>/dev/null | grep -A 200 "===== OUTPUT ====="
```

Sortie

```
francy@Ubuntu:~/Documents/TSP_S2/Cassiopee/CloudSIM-TSP/modules/cloudsim-examples$ cd ~/Documents/TSP_S2/Cassiopee/CloudSIM-TSP
mvn clean install -DskipTests -q && cd modules/cloudsim-examples && mvn exec:java -Dexec.mainClass="org.cloudbus.cloudsim.exempl
es.CloudSimExample3" 2>/dev/null | grep -A 200 "===== OUTPUT ====="
===== OUTPUT =====
Cloudlet ID   STATUS   Data center ID   VM ID   Time   Start Time   Finish Time
1            SUCCESS    2              1       80     0,01         80,01
0            SUCCESS    2              0      160     0,01        160,01
CloudSimExample3 finished!
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 0.963 s
[INFO] Finished at: 2026-03-16T17:09:22+01:00
[INFO] -----
```

Interpretation

VM2 est 2 fois plus puissante que VM1 et donc finit 2 fois plus vite :

```
40 000 ÷ 500 = 80 sec
40 000 ÷ 250 = 160 sec
```

Remarques

Pour cet exemple, les variations utiles seraient juste de rejouer ce qu'on a déjà fait sur l'exemple1 (changer MIPS, length...) : **on ne découvrirait rien de nouveau.**

Ce qu'Example3 apportait de nouveau c'était :

- 2 hôtes dans le datacenter
- 2 VMs de puissances différentes

Ces deux points sont compris et confirmés par les résultats ci-dessus.

CloudSimExample4.java

Exemple de simulation avec 2 datacenters distincts, chacun avec un hôte en **SpaceShared**, pour observer la répartition des VMs sur plusieurs datacenters.

Ce fichier :

- Crée 2 Datacenters distincts (Datacenter_0 et Datacenter_1) chacun avec 1 hôte à 1000 MIPS.

Ici, les hôtes utilisent **VmSchedulerSpaceShared** au lieu de TimeShared. Donc chaque VM obtient un cœur dédié.

- Configure 2 VMs identiques à 250 MIPS avec **CloudletSchedulerTimeShared**.
- Crée 2 Cloudlets identiques de 40 000 MI liés chacun à une VM.
- Le broker est créé via une méthode dédiée createBroker()

Sortie

```
francy@Ubuntu:~/Documents/TSP_S2/Cassiopee/CloudSIM-TSP/modules/cloudsim-examples$ cd ~/Documents/TSP_S2/Cassiopee/CloudSIM-TSP
&& mvn clean install -DskipTests -q && cd modules/cloudsim-examples && mvn exec:java -Dexec.mainClass="org.cloudbus.cloudsim.exa
mples.CloudSimExample4" 2>/dev/null | grep -A 200 "===== OUTPUT ====="
===== OUTPUT =====
Cloudlet ID   STATUS   Data center ID   VM ID   Time   Start Time   Finish Time
0             SUCCESS   2                0       160    0,02         160,02
1             SUCCESS   3                1       160    0,02         160,02
CloudSimExample4 finished!
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 0.857 s
[INFO] Finished at: 2026-03-16T17:37:40+01:00
[INFO] -----
```

Exemples : 5, 6, 7, 8 et 9

Ces tests n'ont pas été exécutés, car ils sont indirectement couverts à travers les variations de paramètres et de politiques d'ordonnancement

présentées dans les exemples précédents (notamment l'Exemple 9, qui compare TimeShared et SpaceShared). Leur exécution n'aurait donc apporté aucun élément d'analyse supplémentaire.

Cette remarque vaut pour certains cas ; toutefois, pour d'autres (notamment l'Exemple 8, qui décrit un GlobalBroker personnalisé créant un nouveau broker à $t = 200$ pendant la simulation), leur utilité immédiate n'apparaissait pas clairement.

Autres Exemples : Mesure de l'energie

Structure des exemples dans le dossier power/

planetlab/ : Charges réelles issues de traces PlanetLab (réseau mondial de serveurs)

- **Caractéristiques :**
 - traces CPU collectées sur des serveurs réels
 - simulation réaliste
 - durée typique : 24 heures
 - utilisées dans les articles scientifiques

- **But :**

Tester les algorithmes d'optimisation énergétique dans un environnement proche du réel.

__random/ : Charges aléatoires synthétiques (Utilisation du CPU generé aléatoirement)

- **Caractéristiques :**
 - utilisation CPU générée par un modèle aléatoire
 - plus rapide à simuler
 - utile pour tests et comparaison rapide
- **But :**
 - valider le comportement d'un algorithme sans dépendre de traces externes

Les fichiers communs aux deux sont :

1. NonPowerAware : Simulation sans gestion d'énergie

- **Caractéristiques :**
 - aucune consolidation de VM
 - aucune migration
 - aucun arrêt de serveur

Tous les hôtes restent actifs.

- **But :**

Servir de baseline (référence).

Ensuite on compare :

Energy avec optimisation
VS
Energy sans optimisation

2. Dvfs : Dynamic Voltage Frequency Scaling

Les noms de fichiers suivent un pattern :

Structure des noms de fichiers

[Algorithme][Politique de migration]

Algorithmes de détection de surcharge

Iqr = InterQuartile Range
Lr = Local Regression
Lrr = Local Robust Regression
Mad = Median Absolute Deviation
Thr = Static Threshold

Quand surcharge détectée --> Migration de VM

1. Iqr — InterQuartile Range

Principe : Analyse statistique de l'utilisation CPU

Si utilisation CPU dépasse :

$Q3 + k \times IQR$ avec : $IQR = Q3 - Q1$

alors :

host considéré comme surchargé

Avantage : adaptatif aux variations de charge

2. Lr — Local Regression

Principe : régression linéaire sur l'historique CPU

Il prédit l'utilisation CPU future.

Si surcharge prévue ---> migration anticipée

3. Lrr — Local Robust Regression

Version robuste de LR.

Différence : Moins sensible aux valeurs extrêmes (outliers)

(A Utiliser quand la charge CPU instable)

4.Mad — Median Absolute Deviation

Il Mesure la **dispersion autour de la médiane**.

Si utilisation CPU dépasse :

$\text{median} + k \times \text{MAD}$

alors :

host surchargé

5. Thr — Static Threshold

C'est une Méthode simple.

Exemple :

$\text{CPU} > 80\%$

alors :

host surchargé

Inconvénient : non adaptatif

Politiques de migration

Quand un host est surchargé, quelle VM migrer ?

Mc = Minimum Correlation
Mmt = Minimum Migration Time
Mu = Minimum Utilization
Rs = Random Selection

1. Mc : Minimum Correlation

Principe :

- On Mesure la corrélation CPU entre VMs...
- Migration de la VM la moins corrélée avec les autres

- **But** : réduire risque de surcharge future

2. Mmt : Minimum Migration Time

On Choisit la VM la plus rapide à migrer

3. Mu : Minimum Utilization

On Choisit la VM avec la plus faible utilisation CPU

Idée : Migrer une petite charge, plutôt qu'une grosse

4. Rs : Random Selection

Sélection d'une VM choisie aléatoirement.

/RANDOM

Exemple 1 : NonPowerAware.java

NonPowerAware implique que Le système gère les machines virtuelles, CPU, mémoire, etc...Sans chercher à optimiser la consommation électrique.

Description et analyse

Simulation d'un datacenter qui consomme toujours à puissance maximale (Pas de gestion d'énergie). C'est la référence de base pour comparer avec les autres politiques.

Voici ce qui change par rapport aux exemples précédents :

1. PowerDatacenterNonPowerAware

```
PowerDatacenterNonPowerAware datacenter =  
Helper.createDatacenter("Datacenter",  
  
PowerDatacenterNonPowerAware.class, ...)
```

Tous les hôtes consomment 100% de leur puissance en permanence, même à vide.____

2. Paramètres externalisés dans des classes dédiées

```
RandomConstants.NUMBER_OF_VMS           // nb de VMs → dans  
RandomConstants.java  
  
RandomConstants.NUMBER_OF_HOSTS         // nb d'hôtes  
  
Constants.SIMULATION_LIMIT              // durée de simulation → dans  
Constants.java  
  
Constants.ENABLE_OUTPUT                  // activer/désactiver les logs
```

NB : Ici, On ne modifiera plus le code directement, seules les constantes le seront.

3. Limite de simulation :

```
CloudSim.terminateSimulation(Constants.SIMULATION_LIMIT);
```

La simulation s'arrête à un temps fixé et pas quand tous les Cloudlets sont finis.

4. La Sortie est plus enrichie

```
javaHelper.printResults(datacenter, vmList, lastClock, ...)
```

Affiche énergie consommée + métriques de performance : **c'est la nouveauté principale !**

Scenario

- 50 hôtes physiques (mix HP G4 et G5)
- 50 VMs (mix des 4 types EC2)
- Charges CPU aléatoires mais reproductibles
- Simulation sur 24 heures

On mesure donc l'énergie totale consommée sur 24h

Commande

```
cd ~/Documents/TSP_S2/Cassiopee/CloudSIM-TSP && mvn clean install -
DskipTests -q && cd modules/cloudsim-examples && mvn exec:java -
Dexec.mainClass="org.cloudbus.cloudsim.examples.power.random.NonPowerAware"
2>/dev/null
```

Avec Filtre (Pour ne voir que l'essentiel) :

```
cd ~/Documents/TSP_S2/Cassiopee/CloudSIM-TSP && mvn clean install -
DskipTests -q && cd modules/cloudsim-examples && mvn exec:java -
Dexec.mainClass="org.cloudbus.cloudsim.examples.power.random.NonPowerAware"
2>/dev/null | grep -E "Energy|migration|SLA|hosts|VMs|Experiment|simulation
time"
```

Sortie



Interprétation

La sortie se divise en 3 parties :

- Partie 1 : les logs détaillés (les Très longues lignes)
- Partie 2 : Le résumé final

```
Experiment name: random_npa
Number of hosts: 50
Number of VMs: 50
Total simulation time: 600,01 sec    --> Durée simulée
Energy consumption: 1,05 kWh         --> Energie Consommée
Number of VM migrations: 0          --> nb de migrations effectuées
Number of host shutdowns: 50        --> nb de serveurs éteints
```

- Partie 3 : Les métriques SLA

```
SLA: NaN%
SLA perf degradation: NaN%
SLA time per active host: 0,00%    --> % du temps où SLA est violé
```

```
Overall SLA violation: NaN%      --> violation globale
Average SLA violation: 0,00%     --> violation moyenne
```

SLA (Service Level Agreement) = Engagement de performance envers l'utilisateur.

NaN (Not a Number) : Pas calculable car les Cloudlets n'ont pas de SLA défini ici.

Exemple 2 : DVFS.java

Simulation d'un datacenter avec DVFS activé. Ici le serveur réduit sa fréquence CPU (et donc sa consommation) quand la charge est faible.

Ce qui est nouveau par rapport à NonPowerAware est :

```
String vmAllocationPolicy = "dvfs";
```

Tout le reste (50 VMs, 50 hôtes, mêmes charges) est identique. Au fait, c'est voulu pour comparer uniquement l'impact du DVFS.

Fonctionnement

NonPowerAware : Hôte à 10% de charge -> consomme quand même 100% de puissance

DVFS : Hôte à 10% de charge -> réduit la fréquence CPU -> consomme proportionnellement moins

Execution

```
cd ~/Documents/TSP_S2/Cassiopee/CloudSIM-TSP && mvn clean install -
DskipTests -q && cd modules/cloudsim-examples && mvn exec:java -
Dexec.mainClass="org.cloudbus.cloudsim.examples.power.random.Dvfs"
2>/dev/null | grep -E "Energy|migration|SLA|hosts|VMs|Experiment|simulation
time"
```

Sortie

```
Experiment name: random_dvfs
Number of hosts: 50
Number of VMs: 50
Total simulation time: 600,01 sec
Energy consumption: 0,17 kWh
Number of VM migrations: 0
SLA: NaN%
SLA perf degradation due to migration: NaN%
SLA time per active host: 0,00%
Overall SLA violation: NaN%
Average SLA violation: 0,00%
Mean time before a VM migration: NaN sec
StDev time before a VM migration: NaN sec
```

DVFS vs NonPowerAware

Résultats côte à côte			
Métrique	NonPowerAware	DVFS	Gain
Énergie	1,05 kWh	0,17 kWh	-84%
Migrations	0	0	=
SLA violation	0%	0%	=
Durée simulation	600 sec	600 sec	=

Autres exemples dans [power/random/](#)

Les autres exemples [power/random/](#) ajoutent des **politiques de migration de VMs par-dessus le DVFS**.

Par exemple :

- IqrMc = DVFS + migration IQR
- ThrMc = DVFS + migration seuil
- (...)

Et ainsi de suite.

Exemple 1

DVFS + migration de VMs avec deux politiques combinées :

```
javavmAllocationPolicy = "iqr";           // détection de surcharge par IQR
vmSelectionPolicy      = "mc";             // sélection de la VM à migrer par
MC
parameter              = "1.5";           // seuil de sécurité IQR
```

Explication :

- **IQR - Inter Quartile Range (allocation)** : Détecte quand un hôte est surchargé en analysant l'historique de sa charge CPU. Si la charge dépasse **médiane + 1.5 × écart interquartile**, alors l'hôte est considéré surchargé et les **migrations déclenchées**.
- **MC - Maximum Correlation (sélection)** Quand un hôte est surchargé, choisit quelle VM migrer en sélectionnant celle dont **la charge CPU est la plus corrélée** avec les autres VMs, pour réduire au maximum la charge de l'hôte.

NB : Les migrations consolident les VMs sur moins d'hôtes (Effet positif) **Mais** elles-mêmes ont un coût énergétique.

Exécution

```
cd ~/Documents/TSP_S2/Cassiopee/CloudSIM-TSP && mvn clean install -
DskipTests -q && cd modules/cloudsim-examples && mvn exec:java -
Dexec.mainClass="org.cloudbus.cloudsim.examples.power.random.IqrMc"
2>/dev/null | grep -E "Energy|migration|SLA|hosts|VMs|Experiment|simulation
time"
```

Sortie

```
Experiment name: random_iqr_mc_1.5
Number of hosts: 50
Number of VMs: 50
Total simulation time: 600,01 sec
Energy consumption: 0,38 kWh
Number of VM migrations: 0
SLA: NaN%
SLA perf degradation due to migration: NaN%
SLA time per active host: 0,00%
Overall SLA violation: NaN%
Average SLA violation: 0,00%
Mean time before a VM migration: NaN sec
StDev time before a VM migration: NaN sec
```

Observation surprenante - 0 migrations et Surconsommation

IqrMc est censé faire des migrations mais on obtient 0 migrations. Pourquoi ?

Hypothèse : La simulation dure seulement 600 sec Et IQR analyse l'HISTORIQUE de charge CPU pour détecter une surcharge.

Il y'aurait donc pas assez d'historique pour déclencher une migration.

C'est une limitation de notre scénario random avec la durée courte. Les migrations apparaîtront **peut être** dans les scénarios planetlab qui ont de vraies traces de charge sur 24 heures.

Pourquoi IqrMc consomme PLUS que DVFS ? DVFS -> 0,17 kWh : adapte la fréquence directement
IqrMc -> 0,38 kWh : DVFS + overhead de calcul IQR sans bénéfice des migrations

Sans migrations effectives, IqrMc **ajouterait** du calcul sans consolidation. Donc, plus d'énergie que DVFS seul.

/PLANETLAB

NonPowerAware.java

Une seule différence majeure avec celui dans **/random** est **la source de charge**. En effet :

- **random/** : charge aléatoire synthétique

```
List<Cloudlet> cloudletList = RandomHelper.createCloudletList(...)
```

- **planetlab/** : vraies traces de charge réelles (réseau mondial de serveurs universitaires réels)

```
String inputFolder = ".../workload/planetlab/20110303"  
List<Cloudlet> cloudletList =  
PlanetLabHelper.createCloudletListPlanetLab(brokerId, inputFolder)
```

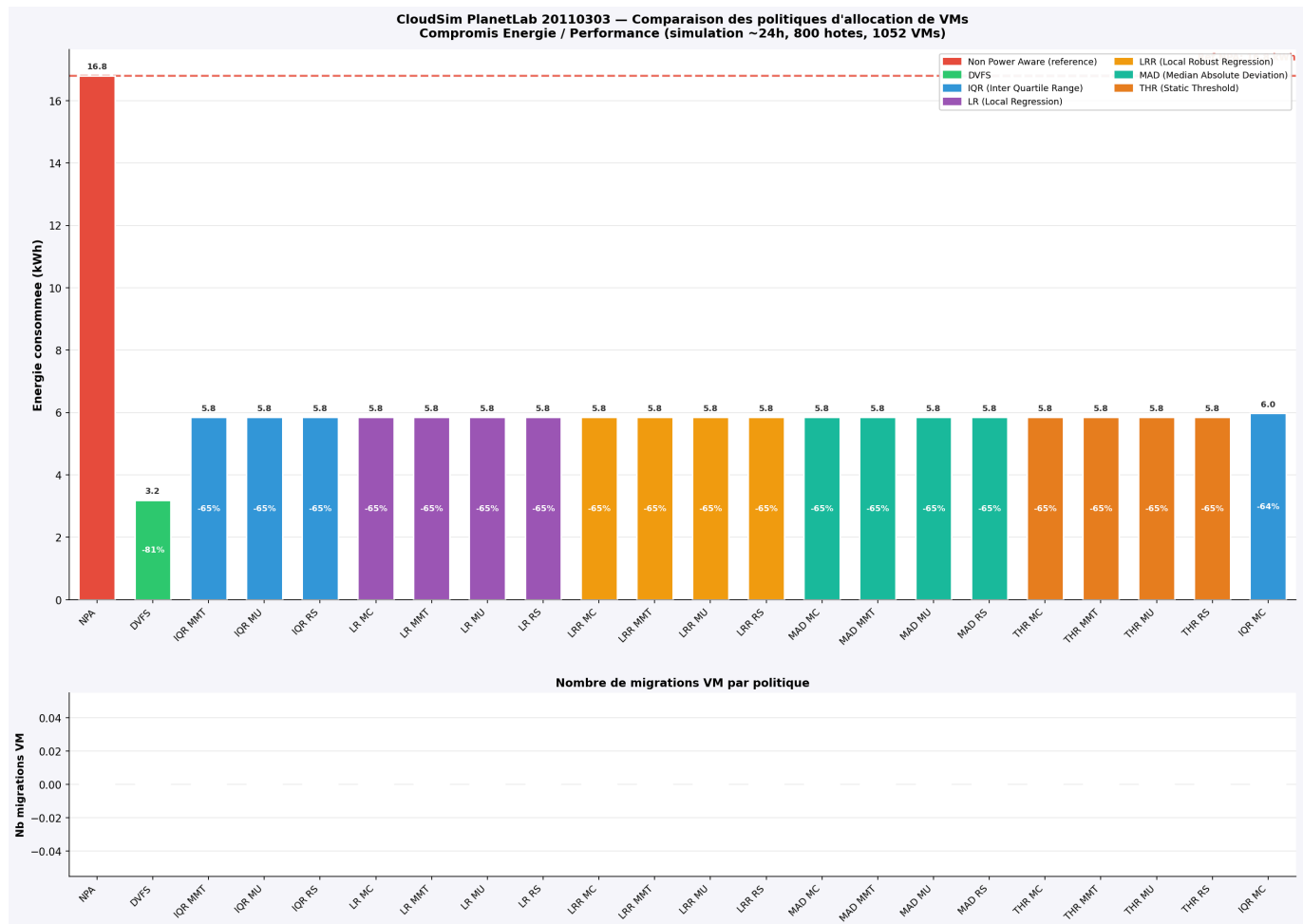
A savoir : - 20110303 = traces du 3 mars 2011 - La charge CPU réelle est mesurée toutes les 5 minutes sur 24 heures.

Execution

```
Experiment name: planetlab_npa  
Number of hosts: 800  
Number of VMs: 1052  
Total simulation time: 600,01 sec  
Energy consumption: 16,80 kWh  
Number of VM migrations: 0  
SLA: NaN%  
SLA perf degradation due to migration: NaN%  
SLA time per active host: 0,00%  
Overall SLA violation: NaN%  
Average SLA violation: 0,00%  
Mean time before a VM migration: NaN sec  
StDev time before a VM migration: NaN sec
```

Remarques : Aucune migration, malgré la variance des paramètres.

Synthese de planetLab



Interprétation

Sans **migrations effectives**, toutes ces politiques font exactement la même chose :

- DVFS seul -> adapte la fréquence -> 3,2 kWh
- IQR/LR/MAD/THR + X -> DVFS + overhead de calcul -> 5,8 kWh

X = On migre qui ?

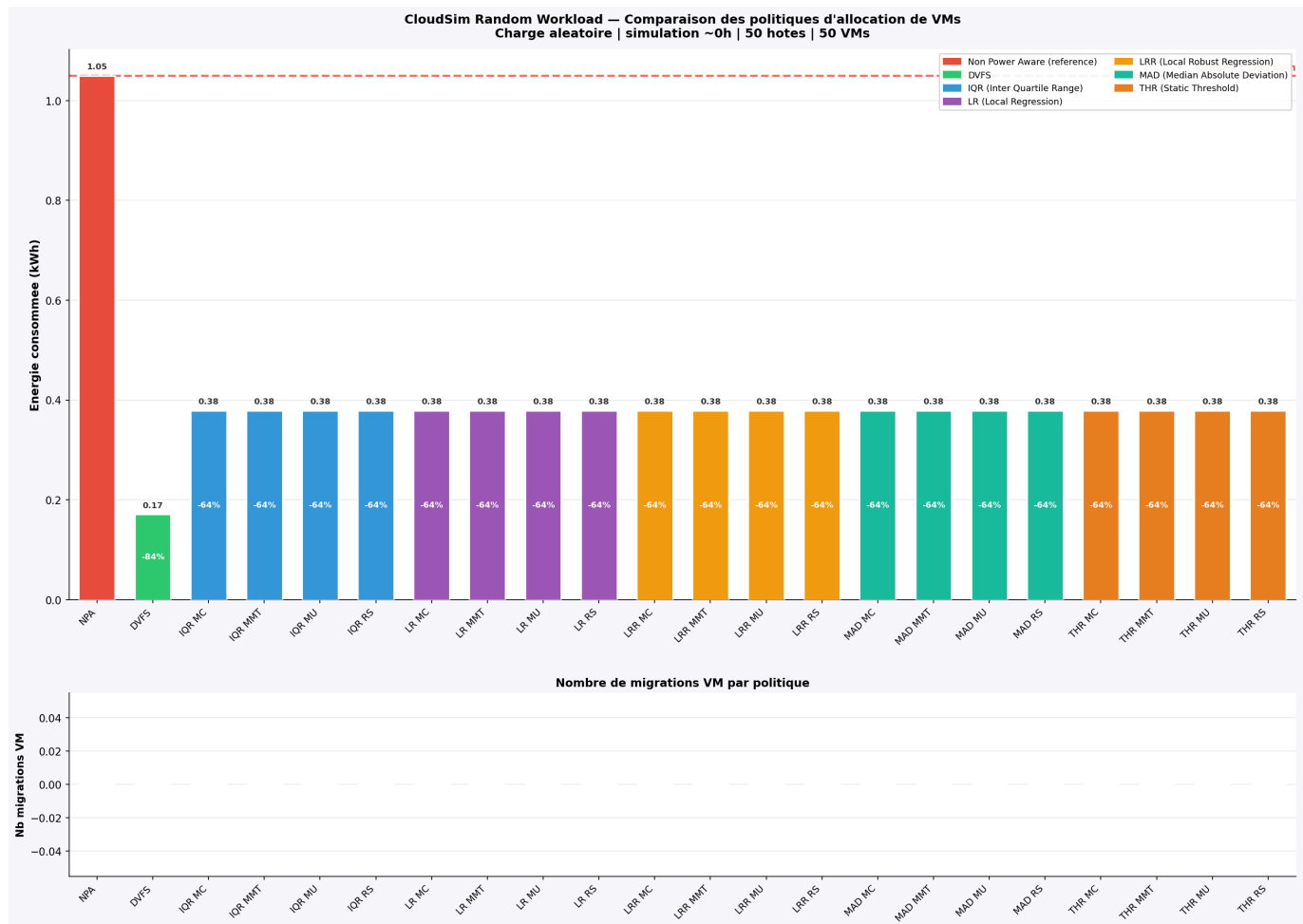
L'overhead de calcul est quasi identique pour toutes, ce qui conduit au même résultat.

Conclusion

Dans notre simulation, le **DVFS seul** est la politique la plus efficace car les migrations ne se déclenchent pas. Mais dans un vrai datacenter avec des charges plus variables, les migrations permettraient de consolider les VMs et d'éteindre des serveurs, l'énergie devrait donc être encore plus basse.

Synthese de random

- 50 VM
- 15 hotes



Remarque importante - Limitation CloudSim 7G

Par défaut, CloudSim 7G arrête la simulation prématurément (~600 sec au lieu de 86 400 sec) car le `CloudletSchedulerDynamicWorkload` calcule une demande de 0 MIPS quand la charge CPU d'une trace est nulle :

```
double totalMips = getTotalUtilizationOfCpu(getPreviousTime()) *
getTotalMips();
// → si utilisation = 0% → totalMips = 0 → VM inactive → hôte éteint →
simulation terminée
```

Correction appliquée dans :

`modules/cloudsim/src/main/java/org/cloudbus/cloudsim/CloudletSchedulerDynamicWorkload.java`

Remplacer :

```
double totalMips = getTotalUtilizationOfCpu(getPreviousTime()) *
getTotalMips();
```

Par :

```
double totalMips = Math.max(getTotalUtilizationOfCpu(getPreviousTime()) *
    getTotalMips(), 1.0);
```

Ceci force un minimum de 1 MIPS. Donc, les VMs restent actives toute la simulation.

Note :

Cette correction ne s'applique pas à NonPowerAware qui utilise `PowerDatacenterNonPowerAware`, sa simulation reste donc limitée à 600 sec.

Remarques : Migration Impossible ...ou presque !

Dans cette partie, malgré nos efforts pour forcer les migrations au niveau des exemples de CloudSim, notamment en faisant varier tous les paramètres imaginables (MIPS, taille des tâches, nombre de cœurs, seuils, etc.), aucune migration n'a été observée : le nombre de migrations restait toujours égal à 0.

1. Formule utilisée par CloudSim

L'idée de base est :

$$Utilisation_{CPU} = \frac{MIPS_{utilisés_par_VMs}}{MIPS_{totaux_du_Host}}$$

2. Comment CloudSim calcule "MIPS utilisés par les VMs"

Chaque VM :

- a une capacité CPU (ex: 800 MIPS)
- consomme une fraction selon son workload



Donc pour une VM :

$$MIPS_{VM} = MIPS_{alloués} \times UtilizationModel$$

Nous avons d'abord pensé que les charges de travail aléatoires ou celles issues de PlanetLab étaient malgré tout trop faibles. Après avoir identifié les workloads utilisés, nous les avons donc modifiés dans l'espoir de résoudre le problème, mais sans succès.

Nous avons alors entrepris de concevoir notre propre code afin d'imposer des migrations et de vérifier si celles-ci étaient réellement possibles avec notre version de CloudSim. Après un long travail, nous avons finalement réussi à mettre au point un programme permettant d'observer des migrations.

Nous avons ainsi constaté que les migrations fonctionnent bien sur CloudSim, mais que, pour une raison encore inconnue, elles ne s'exécutent pas dans les exemples officiels, comme le montre la sortie ci-dessous.

The screenshot displays an IDE interface for a project named 'CloudSIM-TSP'. The 'Project' view on the left shows a directory structure with 'random' and 'web' folders. The 'web' folder contains 'CloudSimExample1' through 'CloudSimExample4'. The 'random' folder contains 'Constants', 'Helper', 'MigrationExample', and 'RunnerAbstract'. The 'MigrationExample.java' file is open in the editor, showing the following code:

```
13 public class MigrationExample {
54
55     private static final int VM_MIPS = 300; 2 usages
56     private static final int VM_PES = 1; 2 usages
57     private static final int VM_RAM = 1024; 1 usage
58     private static final long VM_BW = 1000; 1 usage
59     private static final long VM_SIZE = 10000; 1 usage
60
61     // =====
```

The 'Run' console shows the following output:

```
30.009999999999998: CloudInformationService: Notify all CloudSim entities for shutting down.
30.009999999999998: Datacenter is shutting down...
30.009999999999998: Broker is shutting down...
Simulation completed.
Simulation completed.

===== RESULTS =====
Simulation time = 30,01 sec
Migrations = 1
Energy = 1,23 Wh

Process finished with exit code 0
```

The 'Performance' panel on the right shows 'Show Results' and a timer at '00:00'. The status 'Process Terminated' is displayed.